

In one line: In the **Swift 6 language mode** data-race safety is a **compile-time guarantee**: all mutable state belongs to one **isolation domain** (an actor, a global actor like `@MainActor`, or `nonisolated`); a value crosses domains only if it is **Sendable** or provably **never used again** by the sender (region isolation / sending). Swift 5 warnings become **errors**.

How it works

- **Domains.** Each actor instance = one domain (serial executor). A **global actor** (`@MainActor`, or `@globalActor` actor DB { static let shared = DB() }) = one domain shared by everything marked with it. **nonisolated** = no domain: sees only let/Sendable state. Crossing = await.
- **Sendable** = marker protocol, safe to share. Implicit for **non-public** structs/enums with Sendable members; **public** ones must declare it. A final class with only let Sendable stored properties conforms (checked). Actors and `@MainActor` classes are Sendable for free. `@unchecked Sendable` = “trust me, I lock”.
- **@Sendable closure** may run concurrently: Sendable captures only, no captured var. (`addTask`, `Task.detached`.)
- **Region-based isolation** (SE-0414, Swift 6.0): a *non*-Sendable value in a **disconnected region** (fresh, not reachable from any actor's state) may be sent if the sender never uses it again. **sending** (SE-0430) spells it in a signature: `f(_ x: sending Box), → sending T`.
- **Globals** (SE-0412): a global/static var must be global-actor isolated, or a Sendable let. Hatch: `nonisolated(unsafe)`.
- **@preconcurrency import M** silences Sendable errors for types of a not-yet-migrated module.
- **Swift 6.2 / Xcode 26 “Approachable Concurrency”**: default isolation `MainActor` (SE-0466); `nonisolated async` runs on the caller's actor (SE-0461), `@concurrent` opts out; isolated conformance extension `V: @MainActor P` (SE-0470).

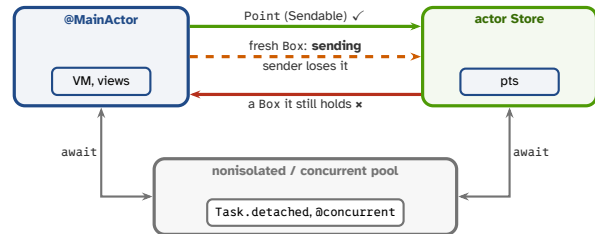
Example

```
final class Box { var n = 0 } // not Sendable
struct Point: Sendable { var x, y: Int } // value: fine
actor Store { var pts: [Point] = []
  func add(_ p: Point) { pts.append(p) }
  func take(_ b: sending Box) { b.n += 1 } }
@MainActor final class VM { // UI domain
  let store = Store()
  func run() async {
    await store.add(Point(x: 1, y: 2)) // Sendable crosses
    let b = Box() // disconnected region
    await store.take(b) // OK: sent away...
    // b.n = 5 error: 'b' used after being sent
  }
  nonisolated func id() -> String { "vm" } // no hop
}
```

Migration strategy

1. Swift 5 mode + **complete** checking as *warnings*:
`SWIFT_STRICT_CONCURRENCY = complete (SwiftPM .enableUpcomingFeature("StrictConcurrency"))`.
2. **One module at a time**: UI types `@MainActor`, models as Sendable values, deps behind `@preconcurrency import`.
3. Flip it: `SWIFT_VERSION = 6 / swiftLanguageModes: [.v6]`; modules in 5 and 6 mode link together. Then shrink every `@unchecked / (unsafe) / @preconcurrency` — each is debt.

Domains and what may cross



Inside a domain: synchronous. Across: await + the value is **Sendable** or a **disconnected region**. Non-Sendable state still reachable from its home (red) never leaves.

Common errors → fix

Compiler says (abridged)	Fix, best first
Capture of 'x' with non-sendable type 'X' in a '@Sendable' closure Sending 'x' risks causing data races	make X a Sendable value; capture a copy [v = x.v]; isolate both to one actor don't touch x after the send; make it Sendable; param sending mark the caller @MainActor; or make it async + await
Main actor-isolated ... can not be referenced from a nonisolated context Static property 'shared' is not concurrency-safe because it is nonisolated global shared mutable state	static let + Sendable type; @MainActor static var; last resort nonisolated(unsafe)
Stored property 'n' of 'Sendable'-conforming class 'C' is mutable Main actor-isolated method cannot satisfy nonisolated protocol requirement	let; make it an actor; or Mutex/lock + @unchecked Sendable nonisolated + MainActor.assumeIsolated; @MainActor protocol; 6.2 isolated conformance

Traps

- ! **Swift 6 compiler ≠ Swift 6 mode**: an old project on the 6.x compiler still builds in Swift 5 mode; mode is per module.
- ! `@unchecked Sendable` over an unlocked var mutes the checker and *keeps* the race.
- ! `Task.detached` in an actor is **not** isolated to it; `Task {}` inherits the actor.
- ! Actors stop *data* races, not *logic* races — state can change across any await (reentrancy).
- ! A public struct is **not** implicitly Sendable.

Remember

Every value has one home. It may travel if it is the 3 S:
Sendable, **Sent** (disconnected, never touched again), or
Synchronised by you (`@unchecked`, at your own risk).

Related: concurrency (async/await, structured concurrency) · actors & reenrancy (`@MainActor` UI confinement) · `AsyncSequence` & `AsyncStream` (continuations) · locks & synchronisation primitives (Mutex, iOS 18)